

ECIP Technical Architecture Documentation (Messenger Community Intelligence Platform - ECIP)

Table of Contents

- 1. 1. Architecture Overview 1
- 2. 2. Component & Service Breakdown 5
 - 2.1. 2.1 Component Responsibilities 5
- 3. 3. Data & Event Flow 5
- 4. 4. Technology Stack & Guardrails 6
 - 4.1. 4.1 Stack Overview 6
 - 4.2. 4.2 Guardrails 7
- 5. 5. Module & Repository Structure 7
- 6. 6. Security, Observability & Deployment 7
 - 6.1. 6.1 Security 7
 - 6.2. 6.2 Observability 8
 - 6.3. 6.3 Deployment 8
- 7. 7. Traceability & References 8
- 8. 8. Guardrails & Open Points 8
 - 8.1. 8.1 Guardrails 8
 - 8.2. 8.2 Open Points 9

1. 1. Architecture Overview

The ECIP platform connects to Telegram as a real user (not a bot), listens in various groups, channels, and discussion chats, collects information passively, and can later interact with other users. This approach enables deep integration and context awareness, going beyond the limitations of the Telegram Bot API.

```
Failed to generate image: undefined method `to_sym' for nil:NilClass
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml

Person(admin, "Admin", "Manages and configures ECIP")
Person(user, "Telegram User (Human)", "End user in Telegram groups/channels")
```

```

System(ecip, "ECIP Platform (Telegram User)", "Connects as a real Telegram user,
listens in groups/channels/discussion chats, collects information, interacts later")
System_Ext(telegram, "Telegram", "Telegram Messenger Platform")
System_Ext(groups, "Telegram Groups/Channels/Discussions", "Various group, channel,
and discussion chat contexts on Telegram")

Rel(admin, ecip, "Configures, Monitors")
Rel(ecip, telegram, "Connects as Telegram User via TDLib")
Rel(ecip, groups, "Listens, collects messages/events, interacts")
Rel(user, groups, "Participates, posts messages")
Rel(groups, telegram, "Exist and are managed on Telegram")

SHOW_LEGEND()
@enduml

```

```

Failed to generate image: undefined method `to_sym' for nil:NilClass
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-
PlantUML/master/C4_Container.puml

System_Boundary(ecip, "ECIP Platform") {
    Container(tdlib, "TDLib Adapter", "Java/Spring Boot", "Telegram client integration,
event normalization")
    Container(context, "Conversation Context Service", "Java/Spring Boot", "Tracks
threads, speaker roles")
    Container(intent, "Intent Classifier", "Java/Spring Boot", "Rule-based intent
classification")
    Container(policy, "Policy Engine", "Java/Spring Boot", "Deterministic policy logic,
escalation")
    Container(llm, "LLM Orchestrator", "Java/Spring Boot", "Model routing, prompt
templates")
    Container(moderation, "Moderation Service", "Java/Spring Boot", "Toxicity filter,
rule violations")
    Container(audit, "Audit/Observability Service", "Java/Spring Boot", "Logging,
tracing, cost tracking")
    Container(admin, "Admin API", "Java/Spring Boot", "Admin endpoints, JWT security")
    Container(db, "PostgreSQL", "Database", "Persistence for all services")
    Container(kafka, "Kafka", "Event Backbone", "Event-driven communication")
}
System_Ext(telegram, "Telegram")
Rel(telegram, tdlib, "TDLib API")
Rel(tdlib, kafka, "Publishes events")
Rel(kafka, context, "Event stream")
Rel(kafka, intent, "Event stream")
Rel(kafka, policy, "Event stream")
Rel(kafka, llm, "Event stream")
Rel(kafka, moderation, "Event stream")
Rel(kafka, audit, "Event stream")
Rel(admin, admin, "Configures, Monitors")
Rel(admin, policy, "Policy configuration")

```

```

Rel(admin, moderation, "Moderation rules")
Rel(admin, audit, "Audit access")
Rel(context, db, "Reads/Writes")
Rel(intent, db, "Reads/Writes")
Rel(policy, db, "Reads/Writes")
Rel(llm, db, "Reads/Writes")
Rel(moderation, db, "Reads/Writes")
Rel(audit, db, "Reads/Writes")
Rel(admin, db, "Reads/Writes")
SHOW_LEGEND()
@enduml

```

Failed to generate image: undefined method `to_sym' for nil:NilClass

```
@startuml
```

```
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml
```

```

Container_Boundary(tdlib, "TDLib Adapter") {
    Component(tdlib_core, "TDLib Core", "C++/JNI", "Telegram protocol integration")
    Component(tdlib_api, "TDLib API", "Java", "Java bindings for TDLib")
    Component(tdlib_events, "Event Publisher", "Java", "Publishes normalized events to Kafka")
}

```

```

Container_Boundary(context, "Conversation Context Service") {
    Component(context_tracker, "Thread Tracker", "Java", "Tracks threads and speaker roles")
    Component(context_api, "Context API", "Java", "Provides context data to other services")
}

```

```

Container_Boundary(intent, "Intent Classifier") {
    Component(intent_rules, "Rule Engine", "Java", "Rule-based intent classification")
    Component(intent_api, "Intent API", "Java", "Provides intent classification results")
}

```

```

Container_Boundary(policy, "Policy Engine") {
    Component(policy_logic, "Policy Logic", "Java", "Deterministic decision logic")
    Component(policy_api, "Policy API", "Java", "Policy configuration and versioning")
}

```

```

Container_Boundary(llm, "LLM Orchestrator") {
    Component(llm_router, "Model Router", "Java", "Routes requests to models")
    Component(llm_templates, "Prompt Templates", "Java", "Manages prompt templates")
}

```

```

Container_Boundary(moderation, "Moderation Service") {
    Component(moderation_filter, "Toxicity Filter", "Java", "Detects rule violations")
    Component(moderation_api, "Moderation API", "Java", "Provides moderation results")
}

```

```

}

Container_Boundary(audit, "Audit/Observability Service") {
    Component(audit_logger, "Logger", "Java", "Structured logging, cost tracking")
    Component(audit_metrics, "Metrics Exporter", "Java", "Prometheus/OpenTelemetry
integration")
}

Container_Boundary(admin, "Admin API") {
    Component(admin_endpoints, "Admin Endpoints", "Java", "Admin operations, JWT
security")
}
@enduml

```

```

Failed to generate image: undefined method `to_sym' for nil:NilClass
@startuml C4_Code
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-
PlantUML/master/C4_Component.puml

Container_Boundary(tdlib_adapter, "telegram-tdlib-adapter") {
    Component(TdlibClient, "TdlibClient", "Handles Telegram connection and events")
    Component(EventPublisher, "EventPublisher", "Publishes events to Kafka")
}

Container_Boundary(conversation_context, "conversation-context") {
    Component(ThreadTracker, "ThreadTracker", "Tracks threads and speaker roles")
}

Container_Boundary(intent_classifier, "intent-classifier") {
    Component(RuleEngine, "RuleEngine", "Rule-based intent classification")
}

Container_Boundary(policy_engine, "policy-engine") {
    Component(PolicyLogic, "PolicyLogic", "Deterministic policy logic")
}

Container_Boundary(llm_orchestrator, "llm-orchestrator") {
    Component(ModelRouter, "ModelRouter", "Routes requests to models")
}

Container_Boundary(moderation_service, "moderation-service") {
    Component(ToxicityFilter, "ToxicityFilter", "Detects rule violations")
}

Container_Boundary(audit_service, "audit-service") {
    Component(Logger, "Logger", "Structured logging")
}

Container_Boundary(admin_api, "admin-api") {
    Component(AdminEndpoints, "AdminEndpoints", "Admin operations, JWT security")
}

```

```

}

Rel(tdlib_adapter, conversation_context, "Provides events for context tracking")
Rel(tdlib_adapter, intent_classifier, "Provides events for intent classification")
Rel(tdlib_adapter, policy_engine, "Provides events for policy decisions")
Rel(tdlib_adapter, llm_orchestrator, "Provides events for LLM orchestration")
Rel(tdlib_adapter, moderation_service, "Provides events for moderation")
Rel(tdlib_adapter, audit_service, "Provides events for auditing")
Rel(admin_api, policy_engine, "Configures policies")
Rel(admin_api, moderation_service, "Configures moderation rules")
Rel(admin_api, audit_service, "Configures audit access")

@enduml

```

2. 2. Component & Service Breakdown

2.1. 2.1 Component Responsibilities

- **TDLib Adapter:** Connects to Telegram as a real user via TDLib, listens in groups/channels/discussion chats, normalizes updates, publishes events to Kafka. Enables both passive information collection and later active interaction with other users.
- **Conversation Context Service:** Tracks threads, speaker roles, enriches context for all observed conversations in Telegram groups/channels/discussions.
- **Intent Classifier:** Rule-based classification of message intent for all collected Telegram messages.
- **Policy Engine:** Deterministic policy logic, escalation, versioning for all actions and responses.
- **LLM Orchestrator:** Model routing, prompt templates, cost control for AI-based responses and analysis.
- **Moderation Service:** Toxicity filter, rule violation detection for all collected and generated content.
- **Audit/Observability Service:** Logging, tracing, metrics, cost tracking for all events and actions.
- **Admin API:** Admin endpoints, JWT security, service-to-service authentication, configuration of Telegram user credentials and group/channel selection.
- **Kafka:** Event backbone for all services, transports all Telegram events and internal actions.
- **PostgreSQL:** Persistent storage for all services, including collected Telegram data and context.

3. 3. Data & Event Flow

Failed to generate image: Could not find the 'mmdc' executable in PATH; add it to the PATH or specify its location using the 'mmdc' document attribute

```

sequenceDiagram
    participant TG as Telegram
    participant ECIP as ECIP Platform (Telegram User)

```

participant GR as Groups/Channels/Discussions
participant TD as TDLib Adapter
participant K as Kafka
participant CS as Context Service
participant IC as Intent Classifier
participant PE as Policy Engine
participant LLM as LLM Orchestrator
participant MOD as Moderation Service
participant AUD as Audit Service
participant ADMIN as Admin API
participant DB as PostgreSQL

ECIP->>TG: Connects as Telegram User via TDLib
ECIP->>GR: Listens, collects messages/events
GR->>ECIP: Receives messages/events
ECIP->>TD: Forwards updates
TD->>K: Publishes MessageReceivedEvent
K->>CS: Event
CS->>K: ContextEnrichedEvent
K->>IC: Event
IC->>K: IntentClassifiedEvent
K->>PE: Event
PE->>K: PolicyDecisionEvent
K->>LLM: Event (if AI needed)
LLM->>K: AIResponseEvent
K->>MOD: Event
MOD->>K: ModerationResultEvent
K->>AUD: All events for logging
Note over DB: All services persist state/data
Note over ADMIN: Configures, Monitors

4. 4. Technology Stack & Guardrails

4.1. 4.1 Stack Overview

- Java 25, Spring Boot 4: Modern, robust, reactive microservices
- Kafka: Event-driven backbone, decoupling, scalability
- PostgreSQL: Reliable, transactional persistence
- TDLib: Full Telegram client integration as a real user (not just Bot API)
- OpenTelemetry, Prometheus: Observability, metrics, tracing
- Vault: Secret management
- JWT: API and service-to-service security
- Maven: Build, dependency management

4.2. 4.2 Guardrails

- All event-driven, no direct service-to-service calls (except Admin API)
- All external communication with Telegram via TDLib as a real user (not Bot API)
- All responses must pass through policy and moderation logic
- No business logic in integration/adaptor layers
- All configuration and secrets via Vault
- All code and infrastructure changes require ADR (see /docs/adr/)
- All technical decisions must be documented (ADR or architecture.adoc)
- No direct DB access from outside services (only via service APIs)
- Observability (logging, tracing, metrics) is mandatory for all services

5. 5. Module & Repository Structure

```
telegram-intelligence-starter/  
├── pom.xml  
├── README.md  
├── docs/  
│   ├── architecture.adoc  
│   ├── adr/  
│   ├── sequence-diagrams/  
│   └── threat-model.md  
├── telegram-core/  
├── telegram-tdlib-adapter/  
├── conversation-context/  
├── intent-classifier/  
├── policy-engine/  
├── llm-orchestrator/  
├── moderation-service/  
├── audit-service/  
└── admin-api/
```

6. 6. Security, Observability & Deployment

6.1. 6.1 Security

- JWT for admin and service-to-service authentication
- Vault for secret management
- All responses pass through policy and moderation logic
- Full auditability and traceability
- All secrets/configs only in Vault, never in code

6.2. 6.2 Observability

- OpenTelemetry for distributed tracing
- Prometheus for metrics
- Structured logging (JSON)
- Audit logs for all decisions and events
- All services must expose health and metrics endpoints

6.3. 6.3 Deployment

- Docker for all services
- CI/CD via Maven, GitHub Actions/Jenkins
- Local development via Docker Compose
- Kubernetes-ready (future)
- All deployments must be reproducible and documented

7. 7. Traceability & References

- All milestones and user stories are mapped to components and phases (see MILESTONES.md, DEEP-DIVE_MILESTONES.md, PHASE-1 to PHASE-5_USER_STORIES.md)
- ADRs (Architecture Decision Records) are referenced but need to be completed for all major decisions (see /docs/adr/)
- Telegram dependency and the ECIP platform's role as a real Telegram user are explicitly modeled in all diagrams
- All referenced documents (MinimalIdeaTechnical.md, ExtendedTechnical.md, etc.) are included or linked here

8. 8. Guardrails & Open Points

8.1. 8.1 Guardrails

- Only event-driven communication (except Admin API)
- All technical decisions must be documented (ADR or architecture.adoc)
- No direct DB access from outside services
- All secrets/configs only in Vault
- Observability is mandatory

8.2. 8.2 Open Points

- Complete missing ADRs for all major architecture and technology decisions
- Regularly update diagrams and documentation as the system evolves
- Review and extend security and observability concepts as needed
- Define and document all external dependencies (besides Telegram)
- Add further C4/Cx diagrams as the system grows